

The Python Workbook

So at this point, you've probably installed python and familiarized yourself (to some degree) with the IDLE. So this is the next big step in becoming a super awesome electromagician like Dr. Jasper: actually learning how to use Python.

The basic form of these lessons is:

title of the lesson

Code to be typed and executed in a program goes here.

>>>Code with 3 arrows is intended to be tested in the console, not a program

Quick comments about the code come after the code explaining things like why the code is displayed in a totally inappropriate and overused font like comic sans (it's to deter you from relying on it too much, so view it at your own risk

Student activities about the code come last, and consist of a mix of questions and prompts. They're formatted like question #number below.

Question #number. Why did David choose such a terrible font for the code?

If you type all of the code into python (copy-pasta won't work because this was originally written in MS word) then you'll be well on your way to becoming a python master! (Success with actual snake charming is not guaranteed.)

Lessons:

- | | |
|----------------------------------|---------------------------|
| 1. Printing | 9. For Loops and lists |
| 2. Variables | 10.Functions |
| 3. Basic math | 11.(maybe classes) |
| 4. String literals & comments | 12.Libraries |
| 5. Format specifiers | 13.Writing text files |
| 6. Input | 14.Operating system stuff |
| 7. Conditional and If statements | |
| 8. While loops | |

1. Printing

```
>>>print("this is a single statement")
>>>print("this is a second statement")
>>>print(10)
>>>print("this is one " + "way to concatenate two strings")
```

So this program introduces the print command. It has parentheses because it's a function. Not much to be said for this one, it takes both strings or numbers. One big thing to note though, is that each line holds just a single statement. The enter key is used by python to tell where one statement ends and another begins.

Question 1. Write an expression that would cause print to fail instead of printing:

2. Variables

```
>>>first = "I'm a little "
>>>print(first)
>>>print(first + str(2) + "pot, short and stout")
```

Variables in python are pretty easy. You don't need to declare a type, it's automatically assigned, and the numbers have virtually infinite precision which is useful. The str() casts(makes the program pretend) that whatever is inside is a string. int(), is usually used when trying to use input to read a number.

Question 2. What happens when you remove the str() around 2?

3. Basic Math

```
>>>a = 7
>>>print(a + 1)
>>>print(a * 5)
>>>print(a ^ 1)
>>>a += 5
>>>print(a)
>>>print(a % 3)
```

Math is math is math is math. The last operator might be unfamiliar. It's called a modulo, and although you've probably seen it before, I doubt you've used it as an operator much. It calculates the remainder when a is divided by 3 in this case. It goes with multiplication and division so far as order of operations. The += is an assignment operator, and it increases a by 5. there are similar ones for addition, subtraction, multiplication and division.

Question 3. what is $8 \% 3$? what is $9 \% 3$? what is $3 \% 9$?

Question 4. If $(a / 10) \% 10$ returns the tens digit of a, how would you find the one's digit?

Question 5. Did you notice that $a ^ 1$ didn't return the first power of a, 7? How do you ACTUALLY do powers in python?

4. String Literals and comments

```
Name = "DavidtheBravid" #because davethebrave was too #basic
```

Yeah this is how you put text in your program. It's not too bad. # is used to mark comments (it goes until the end of the line). We already know quotation marks make strings. You can also use ' or """. There are also escape sequences which behave interestingly. They look like this: \n, \t, \b.

Question 6. What does the \n escape do?

Question 7. How do you place just a regular backslash in a string?

5. Format Specifiers

```
>>>print("This text is %s a word" % "missing")
>>>print("I like putting %d into my sentences in place of words" % 69)
>>>print("This lets you put in a float %f (which has decimals)" %2.5)
>>>print("You can control the number of decimals %.1f" % 2.5)
>>>print("it can pad to a minimum number of characters |%5.1f|" % 2.5)
>>>print("you can also put in two numbers: %d %d" % (1, 2) )
```

yeah... the % format specifier is pretty wonky and a touch outdated, but also really useful once you get the hang of it. It's nice to be able to format tables and other information and makes the formatting of tables and arrays a cinch (plus it's useful for input). Generally, %s is used for strings, %d is used for integers, and %f is used for floats (things with fractional components)

Question 8. What would happen if you put %% into a string?

Question 9. What happens when you specify the precision (number of digits after the decimal) of an integer (%d)?

Question 10. How would you specify that the formatted number should occupy seven characters and have a precision of two digits?

6. Input

```
>>> text = input("This will be the text you're reading as you input:\n")
>>> num = input("but does it store numbers?\n")
```

Input is pretty important. This thing does all the hard work for you. Colons and new lines are your friend for making the prompt legible though.

Question 11. After running this program, try `print(num)` and then `print(num + 5)`. What happened? How can you print both?

7. Conditional and If statements

```
>>> True == False
>>> False != True
>>> True == not False
>>> 5 == 2
>>> 5 > 2
num = int(input("Enter a number greater than 5:\n"))
if num <= 5:
    print("please learn to follow instructions.")
    print("Or math.")
elif (num == 5) == True:
    print("so close")
else:
    print("congratulations! Welcome to the machine!")
```

Okay, so let's look at some patterns here. Conditional logic returns true/false. These functions check the equality of two terms, or the truth of an expression.

"If" statements are interesting. Rather than use a brace or symbol, statements inside of a control structure like this if statement are simply indented after the colon ":". There is also "elif", which performs the "else if" control, and "else" which is the fall back if neither "if" nor an "elif" occur.

Question 12. Is there order of operations on conditional statements, e.g. how would `True == False == True` actually get interpreted?:

Question 13. Is `True > False`? what is `True + 1`? What is `False + 1`? What do `true` and `false` actually represent?

Question 14. Please rewrite the above `elif` conditional statement, `(num == 5) == True`, to remove unnecessary code:

8. While loops

```
i = 1
```

```
while i < 500:
```

```
    print("Line %d:Press ctrl-C to break out of infinite loops" % i)
```

```
    i += 1
```

This is a while loop. While the expression following `while` is true, it'll keep looping through the center portion. Also, `ctrl-c`, or `cmd-c` for macs, is the key combination for exiting a loop, and it's a very worthwhile combination to memorize for that reason.

Question 15. What happens if you type `"while 1:"`? or `"while 2:"`?

9. For loops and lists

```
nums = [1,5,6]
print(nums[0])
for i in range(5,10):
    print(i)
```

This is a list and a for loop in Python. They're straightforward... relatively...

Question 16. Look up append, extend and index. Create a second list, and use it or a single number to modify the original list nums. Write "I'm winning bigly" if you managed to get them to work. If not, don't sweat it, just ask us in lab.

10. Functions

```
def fun(name, num = 7):
    print("Did you say %s was the name?" % name)
    print("And you favorite number is %d" % num)
>>>fun("Billy")
>>>fun("Bob", 5)
>>>fun(num = 75, name = "BillyBob")
```

And this is a function. These are fairly straightforward. Use def to say you're making a function, and then the statements you would like it to execute are indented and directly below. Arguments are passed in the parentheses, and then statements that the function performs are indented below it.

Question 17. What is the default value of num?

11. (Maybe) Classes

```
class wallet:
    cash = 0
    def __init__(self, money = 0):
        self.cash = money
        print("I have a wallet")

    def add(self, money):
        self.cash += money
        print("money safely put away")
```

Type the stuff below into the console directly, not into the program

```
>>>x = wallet()
>>>y = wallet(5)
>>>x.cash
>>>y.cash
>>>x.add(15)
```

Okay, classes are a bit confusing. A class is an outline of an object, and when you call it, it instantiates (in-stan-tshee-ate-s, make a new copy of) an actual

usable object. That's what happened when you called `x = wallet()`. `x` is the object, but `wallet` provided the guideline for creating `x` and how it works.

Really, we just need to know how to instantiate classes, like in `x = wallet()`; and how to modify their variables and use their functions, like in `x.cash` and `x.add()`. The classes themselves will be defined in an imported library

Question 18(optional). Would you have preferred exercises for this section?

12. Libraries

```
import time  
  
i = 0  
  
while 1:  
    print(i)  
    time.sleep(1)
```

Python comes with quite a few helpful definitions and functions. Things like `for`, `and` `while` and `print` are already defined so we don't need to worry about creating these functions or structure whenever we create a program in python. Other libraries add functions and definitions to our program so we can create it more easily.

For example, we could potentially create the clock function, but doing so would require a deft knowledge of C and how to create a function to mark the time that the program actually started running, or how to access that variable automatically created by the program. It might be wiser just to import the `time` library instead of going through that work instead.

Question 19. Will import still work if it is at the bottom of a program?

13. Writing to files

```
weg = open("output.txt", "w")
```

```
weg.write("This writes a line to the file!\n")
```

```
weg.write("Unlike print it doesn't automatically")
```

```
weg.write("write \\n at the end of each line\n")
```

```
weg.write("Also, congratulations because this ")
```

```
weg.write(" is the second to last section")
```

```
weg.close()
```

Files are a pretty basic type of object. If you use `file.open()` it will create a file object of that name. The "w" means write, and so a new file will be created in the same folder of your python program with the specified name (any previous files of the same name (endgame.txt) in that folder will be DELETED).

Question 20. What's a more appropriate name for our file object than `weg`?

14. Operating system things

```
import os
fPath = os.path.abspath(__file__)
path = os.path.dirname(fPath)
print(path)
os.startfile(path + "\\sound.mp3")
```

The os library is pretty useful, but it can be difficult to learn. Fortunately, we're going to stick to really basic things with it, like opening files and stuff.

Question 21. Did your code work on the first try, or did it return an error? Try renaming an mp3 sound, and adding it to the same folder specified by fPath.

Question 22. What term referred to the current working python script?

Question 23. Can startfile be used to open things that aren't mp3? Name another filetype you can use it on:

Congratulations! You can do the actual assignments now!!!

Jk, you're completely done with the first python lab though, and you should probably start the assignments within a week so you don't forget them.